An Extended Guide and Derivation to 2D Motion Profiling for Differential Drivetrains Using Bézier Curves

Contents

1 Introduction

1.1 What is Motion Profiling?

As the name suggests, motion profiling profiles a motion — it derives characteristics about a movement, in most cases velocities and their derivatives.

Motion profiling answers the question: how should a robot move from point A to point B?

- How fast should it go?
- When should it turn?
- When should it start slowing down?

In this case the result is a path plus a time-parameterized velocity and angular velocity profile that respects the robot's physical limits.

1.2 Our Goal: Move Along a Curve

We want to follow a curved path while:

- Respecting maximum speed and acceleration
- Slowing down for turns (based on curvature)
- Hitting specified speeds at key positions (keyframes)

To do that, we describe the path mathematically and then compute how the robot should move along it.

How to read this guide. The material is split into three parts. Section 2 builds the geometry: how to describe a path and how to answer questions about it. Section 3 builds the speed planner on top of that geometry. Section 4 deals with turning the plan into motor commands. Passages in shaded boxes like the one below are optional depth — they justify a result the main text has already stated, and skipping them costs you nothing on a first read.

> **Going deeper: what these boxes are for**
>
> If you have taken (or are taking) a first calculus course, the main thread of this guide should be readable. These boxes are where the second-year material lives: convergence of numerical integration, derivations that need the chain rule in two variables, and the failure modes that only show up in degenerate cases. Come back to them when you want to know **why** rather than **what**.

1.3 The Planning Loop

Before any of the math, here is the entire algorithm. Everything in this guide exists to fill in one of these lines.

```
s ← 0                       // distance traveled so far
t ← 0                       // curve parameter, 0 at the start, 1 at the end

while not finished:
    v_max    ← constant                  // the drivetrain's top speed
    v_accel  ← limit from max acceleration
    v_brake  ← limit from needing to stop at the end
    v_curve  ← limit from how sharp the path is here
    v_interp ← limit from the user's keyframes

    v  ← min(v_max, v_accel, v_brake, v_curve, v_interp)
```

```
    Δs ← v · dt                          // distance to cover this timestep
    t  ← findNextT(s, Δs)                // advance along the curve by Δs
    s  ← s + Δs

    emit( position(t), v, curvature(t) · v )
```

Two observations, both worth holding onto:

1. **Every speed limit is an upper bound, so we take the minimum.** There is no clever arbitration between the five candidates. Each one says "you may not go faster than this"; obeying all of them at once means obeying the smallest.
2. **The hard part is `findNextT`.** We want to move a **distance** Δs, but the curve is described by a **parameter** t, and those are not the same thing. Most of Section 2 is about bridging that gap.

2 Describing the Path

2.1 Representing the Path: Cubic Bézier Curves

There are multiple ways to describe a smooth curve. Cubic Bézier curves use four control points (start, end, and two shaping points), which gives a good balance of simplicity and flexibility, and they are what this guide uses throughout [1]. Their x and y coordinates are

$$x(t) = (1-t)^3 x_0 + 3(1-t)^2 t\, x_1 + 3(1-t)\, t^2 x_2 + t^3 x_3$$
$$y(t) = (1-t)^3 x_0 + 3(1-t)^2 t\, y_1 + 3(1-t)\, t^2 y_2 + t^3 y_3$$

Other common choices — quadratic Béziers, cubic Hermite splines, and Catmull–Rom splines — are compared in Section A. Nothing later in this guide depends on the choice: everything downstream needs only $\boldsymbol{r}(t)$, $\boldsymbol{r}'(t)$, and $\boldsymbol{r}''(t)$, which every one of those representations provides.

2.2 Why t Is Not Distance

The obvious way to drive the curve is to step t forward by a fixed amount each timestep — $t = 0.00, 0.01, 0.02, \ldots$ — and command a constant speed. This does not work, and understanding why motivates the next two sections.

Sample a cubic Bézier at evenly spaced values of t and plot the points. They are **not** evenly spaced along the curve. Where the control points pull the curve into a tight turn, the samples bunch together; on the long straight stretches they spread far apart. The parameter t measures progress through the **formula**, not progress through **space**.

Stepping t uniformly therefore means the robot slows down and speeds up arbitrarily, driven by the algebra rather than by anything physical. We need the opposite: given a distance we want to travel, find the t that gets us there. That requires two tools — a way to compute distance from t (Section 2.3), and a way to invert it (Section 2.4).

2.3 Arc Length: From t to Distance

For a parametric curve $\boldsymbol{r}(t) = (x(t), y(t))$, the arc length from the start up to parameter t is

$$s(t) = \int_0^t \|\boldsymbol{r}'(\tau)\|\, d\tau,$$

the integral of speed $\|\boldsymbol{r}'(\tau)\|$ along the curve. For a cubic Bézier there is no closed-form answer to that integral, so we compute it numerically.

2.3.1 The straightforward version

Chop the curve into N small pieces and add up the straight-line distances between consecutive points:

```
s ← 0
for i in 1..N:
    s ← s + | r(i/N) - r((i-1)/N) |
return s
```

This is five lines of code, needs no theory beyond the distance formula, and it works. Its only problem is cost: getting good accuracy needs N in the hundreds, and we call this function several times per control cycle.

2.3.2 The efficient version: Gaussian quadrature

Gaussian quadrature is a smarter way to spend a fixed budget of samples. Instead of spacing them evenly, it places them at special locations (nodes x_i) and weights them by w_i chosen so that the approximation is as accurate as possible:

$$\int_a^b f(x)\,dx \approx \frac{b-a}{2}\sum_{i=1}^{n} w_i\, f\!\left(\frac{b-a}{2}x_i + \frac{a+b}{2}\right).$$

An n-point Gauss–Legendre rule is exact for all polynomials up to degree $2n-1-n$ samples buying the accuracy of a far denser uniform grid. The $n=2$ rule on $[-1,1]$, for instance, uses

$$x_1 = -\frac{1}{\sqrt{3}}, \quad x_2 = \frac{1}{\sqrt{3}}, \qquad w_1 = w_2 = 1,$$

so that

$$\int_{-1}^{1} f(x)\,dx \approx f\!\left(-\frac{1}{\sqrt{3}}\right) + f\!\left(\frac{1}{\sqrt{3}}\right),$$

which integrates any cubic polynomial exactly. Nodes and weights for $n = 2, \ldots, 5$ are tabulated in Section B; the implementation here uses $n = 5$.

To apply this to arc length, set

$$f(\tau) = \|\boldsymbol{r}'(\tau)\|, \quad a = 0, \quad b = t,$$

so the sample points are $\tau_i = \frac{t}{2}\,x_i + \frac{t}{2}$ and

$$s(t) \approx \frac{t}{2}\sum_{i=1}^{n} w_i\, \|\boldsymbol{r}'(\tau_i)\|.$$

In practice we do not apply this to the whole interval at once. We split $[0, t]$ into $m = 8$ equal panels and run the 5-point rule on each, which costs 40 evaluations of $\|\boldsymbol{r}'\|$ per call — cheap enough for every control cycle, and accurate enough that the residual error sits well below the robot's odometry noise:

$$s(t) \approx \sum_{j=0}^{m-1} \frac{h}{2} \sum_{i=1}^{n} w_i \left\| \boldsymbol{r}'\left(hj + \frac{h}{2}(x_i + 1) \right) \right\|, \qquad h = \frac{t}{m}.$$

> **Going deeper: why one panel is not enough**
>
> For a cubic Bézier the integrand $\|\boldsymbol{r}'(\tau)\|$ is the **square root** of a polynomial, not a polynomial itself, so the "exact for degree $2n - 1$" guarantee does not apply at all — no choice of n makes a single application of the rule exact here.
>
> What the rule still gives is fast convergence for smooth integrands, and that convergence is much faster on short intervals than on long ones. On a long or wiggly curve, a single $n = 5$ panel spanning all of $[0, t]$ leaves error large enough to misplace the robot. Splitting into m panels and applying the rule on each — **composite** quadrature — is the standard fix [2]. Accuracy still degrades near degenerate curves; see Section C.

2.4 From Distance Back to t: Newton–Raphson

Section 2.3 gives us distance from a parameter. The planning loop needs the reverse: given that we want to travel a further Δs, find the new parameter t_{next} with

$$s(t_{\text{next}}) = s_{\text{current}} + \Delta s.$$

Equivalently, find the root of

$$F(t) = s(t) - (s_{\text{current}} + \Delta s).$$

Newton–Raphson does this by repeatedly following the tangent line to the axis. Its derivative is free here: $F'(t) = s'(t) = \|\boldsymbol{r}'(t)\|$, the **parametric** speed of the curve (how fast the point moves as t changes), not the robot's commanded velocity. The iteration is

$$t_{k+1} = t_k - \frac{F(t_k)}{F'(t_k)} = t_k - \frac{s(t_k) - (s_{\text{current}} + \Delta s)}{\|\boldsymbol{r}'(t_k)\|},$$

and in practice each iterate is clamped to $t \in [0, 1]$. We stop when

$$\left| t_{k+1} - t_k \right| < \varepsilon \quad \text{or} \quad \left| s(t_k) - (s_{\text{current}} + \Delta s) \right| < \varepsilon \quad \text{or} \quad k \geq k_{\max},$$

with a tolerance such as $\varepsilon = 10^{-6}$ and a cap such as $k_{\max} = 10$ to avoid infinite loops. If Newton fails to converge within $k_{\max}$ iterations, falling back to bisection — essentially a binary search for the zero — is a robust remedy [2].

> **Going deeper: a lower-tech alternative**
>
> If the quadrature-plus-Newton combination is more machinery than you want, there is a well-trodden alternative: precompute a table of (t, s) pairs at, say, 200 uniform values of t once when the path is created, then answer every later query by binary-searching the table and linearly interpolating between neighbors. It costs a few kilobytes and a one-time setup pass, and several production path-following libraries do exactly this. The tradeoff is a fixed accuracy floor set by the table resolution.

2.5 Curvature: How Sharp Is the Path Here?

For a 2D path $\boldsymbol{r}(t) = (x(t), y(t))$, the **signed** curvature is

$$\kappa(t) = \frac{x'(t)y''(t) - y'(t)x''(t)}{\left(x'(t)^2 + y'(t)^2\right)^{3/2}}, \qquad R(t) = \frac{1}{|\kappa(t)|},$$

where $R(t)$ is the radius of curvature: the radius of the circle that best hugs the path at that point. A straight line has $\kappa = 0$ and $R = \infty$; a hairpin has large $|\kappa|$ and small R.

The sign matters. It encodes the turn direction — positive for a left turn, negative for a right, with the usual counterclockwise convention. The speed limit in Section 3.5 depends only on $|\kappa|$, but the commanded angular velocity is $\omega = v\kappa$, so discarding the sign would leave the robot able to turn only one way.

The denominator vanishes wherever the parametric speed does, and the formula is undefined there. That case, and what to do about it, is covered in Section C.

3 Planning the Speed

With the geometry in hand, we can fill in the five limits from Section 1.3. Each subsection below computes one of them.

3.1 A Kinematics Toolbox

Three of the five limits are built from one equation, so it is worth stating first. From basic kinematics with constant acceleration a:

$$v^2 = v_0^2 + 2a\,\Delta s \quad \Longrightarrow \quad \Delta s = \frac{v^2 - v_0^2}{2a}.$$

- If $a > 0$: Δs is the distance needed to **accelerate** from v_0 up to v.
- If $a < 0$: Δs is the distance needed to **decelerate** from v_0 down to v.

Note the form of this equation: it relates speed to **distance**, with no time in it. That is exactly what a path planner wants, since everything is indexed by arc length.

> **Going deeper: where the equation comes from**
>
> Integrate acceleration over distance rather than over time:
>
> $$a = \frac{dv}{dt} \quad \Rightarrow \quad v\,dv = a\,ds \quad \Rightarrow \quad \int_{v_0}^{v} v\,dv = \int_{0}^{\Delta s} a\,ds \quad \Rightarrow \quad \frac{v^2 - v_0^2}{2} = a\,\Delta s,$$
>
> and rearrange. The middle step uses $ds = v\,dt$, which is just the definition of speed.

3.2 Limit 1: Maximum Speed

The robot's absolute top speed, $v_{\max}$. A constant, measured empirically or taken from the motor's free speed and gear ratio. Nothing to derive.

3.3 Limit 2: Acceleration

The robot's speed can only change by $a_{\max}\,\Delta t$ in one timestep, so

$$v_{\text{accel}} = v(t) + a_{\max}\,\Delta t,$$

where $v(t)$ is the current commanded speed. This is what keeps the profile from demanding an instantaneous jump in velocity that the drivetrain cannot deliver.

3.4 Limit 3: Braking for the End of the Path

This limit guarantees the robot can still decelerate to its exit velocity v_{end} by the time the path runs out. From Section 3.1, the distance needed to slow from $v_{\max}$ to v_{end} at maximum deceleration is

$$d_{\text{decel}} = \frac{v_{\max}^2 - v_{\text{end}}^2}{2 \cdot a_{\max}}.$$

While the remaining distance $d_{\text{remaining}}$ exceeds d_{decel}, braking imposes no limit at all. Once $d_{\text{remaining}} \leq d_{\text{decel}}$, we want the largest speed v from which the robot can still reach v_{end} within $d_{\text{remaining}}$. Applying the same kinematic equation over the remaining distance:

$$v_{\text{end}}^2 = v^2 - 2 \cdot a_{\max} \cdot d_{\text{remaining}} \quad \Longrightarrow \quad v_{\text{brake}} = \sqrt{v_{\text{end}}^2 + 2 \cdot a_{\max} \cdot d_{\text{remaining}}}.$$

This is the deceleration ramp of a trapezoidal profile, expressed as a function of distance-to-go: at $d_{\text{remaining}} = d_{\text{decel}}$ it equals $v_{\max}$, and at $d_{\text{remaining}} = 0$ it equals v_{end}.

3.5 Limit 4: Curvature

The wheels of a differential drive robot have a maximum speed. In a turn, the outer wheel travels faster than the robot's center, so the center must slow down to keep the outer wheel within its limit [3], [4]:

$$v_{\text{curve}(t)} = v_{\max} \cdot \frac{R(t)}{R(t) + \frac{w}{2}},$$

where w is the track width (the distance between the left and right wheels) and $R(t)$ is the radius of curvature from Section 2.5. On a straight path $R \to \infty$ and the limit relaxes to $v_{\max}$; in a hairpin $R \to 0$ and it drives the speed toward zero.

> **Going deeper: deriving the curvature speed limit**
>
> Start from the differential drive kinematic equations [3]:
>
> $$V_l = v - r \cdot \omega \quad \text{and} \quad V_r = v + r \cdot \omega,$$
>
> where v is linear velocity, ω is angular velocity, and $r = w/2$ is the distance from the center of the robot to either wheel. Since $\omega = v \cdot \kappa$ by definition of angular velocity, substituting and factoring gives
>
> $$V_l = v \cdot (1 - r \cdot \kappa), \quad V_r = v \cdot (1 + r \cdot \kappa).$$
>
> Each wheel must stay within its maximum velocity:
>
> $$|v \cdot (1 - r \cdot \kappa)| \leq v_{\max}, \quad |v \cdot (1 + r \cdot \kappa)| \leq v_{\max}.$$
>
> Since $v \geq 0$ we can pull v out of the absolute value and divide:
>
> $$\max(|1 - r \cdot \kappa|, |1 + r \cdot \kappa|) \leq \frac{v_{\max}}{v}.$$
>
> That maximum is always $1 + |r \cdot \kappa|$, so
>
> $$1 + |r \cdot \kappa| \leq \frac{v_{\max}}{v} \quad \Rightarrow \quad v \leq \frac{v_{\max}}{1 + |r \cdot \kappa|}.$$
>
> Finally, substituting $|\kappa| = 1/R(t)$ gives the geometric form:

$$v \le \frac{v_{\max}}{1 + r/R(t)} = v_{\max} \cdot \frac{R(t)}{R(t) + r}.$$

3.6 Limit 5: Keyframes

Keyframes are points along the path where the user fixes the robot's speed — for example, start at 0 m/s, reach 1 m/s halfway, end at 0 m/s. Path editing tools such as *PATH.JERRYIO* [5] expose these through a speed graph, where each keyframe is a distance along the path (the x-axis) paired with a desired speed (the y-axis).

Turning that into a speed limit takes two steps: locating each keyframe on the curve, and interpolating between them.

3.6.1 Locating a keyframe on the path

The editor hands us keyframes as (x, y, v): a point the user clicked on the field and the speed they want there. Everything downstream is indexed by t, so the first job is to turn the point $\boldsymbol{p} = (x, y)$ into a parameter.

The tempting shortcut is to solve $x(t) = x_{\text{kf}}$ for t and ignore y. That is wrong on any path that revisits an x coordinate — an S-curve, a loop, or anything that doubles back — because the equation then has several roots and nothing distinguishes the intended one from a branch of the path meters away. It also fails outright when the keyframe sits slightly off the curve, since $x(t) = x_{\text{kf}}$ may have no solution in $[0, 1]$ at all.

The correct question is a projection: which point of the curve is closest to $\boldsymbol{p}$?

$$t^* = \operatorname*{arg\,min}_{t \in [0,1]} \|\boldsymbol{r}(t) - \boldsymbol{p}\|^2.$$

Given t^* for each keyframe, the arc-length position follows from the quadrature of Section 2.3: $s_i = s(t_i^*)$.

Going deeper: solving the projection

Differentiating the objective gives the stationary condition

$$\frac{d}{dt}\|\boldsymbol{r}(t) - \boldsymbol{p}\|^2 = 2(\boldsymbol{r}(t) - \boldsymbol{p}) \cdot \boldsymbol{r}'(t) = 0,$$

which says the error vector is orthogonal to the tangent — geometrically obvious once you see it. Writing $g(t) = (\boldsymbol{r}(t) - \boldsymbol{p}) \cdot \boldsymbol{r}'(t)$, Newton's method applies again with

$$g'(t) = \|\boldsymbol{r}'(t)\|^2 + (\boldsymbol{r}(t) - \boldsymbol{p}) \cdot \boldsymbol{r}''(t), \qquad t_{k+1} = t_k - \frac{g(t_k)}{g'(t_k)}.$$

For a cubic Bézier, g is a quintic in t and can have up to five roots, so the squared distance is **not** convex and Newton alone will happily converge to the wrong local minimum. The implementation therefore scans a coarse grid (64 samples) for the best basin, refines from there, and discards the refinement if it ends up farther from $\boldsymbol{p}$ than the sampled seed.

3.6.2 Interpolating between keyframes

Now we have pairs (s_i, v_i): desired speeds at arc lengths. For an intermediate position s with $s_i \le s \le s_{i+1}$, define the fraction of the way through the interval

$$\lambda = \frac{s - s_i}{s_{i+1} - s_i} \in [0, 1].$$

The obvious move is to interpolate v itself linearly in s. Do not. Along the path $a = v\,\frac{dv}{ds}$, so a profile linear in s implies

$$a(s) = v(s) \cdot \frac{v_{i+1} - v_i}{s_{i+1} - s_i},$$

which is proportional to the current speed: the segment demands the **most** acceleration exactly where the robot is already moving fastest, and the demand changes continuously across an interval the planner treats as one piece.

Interpolating in v^2 instead fixes this:

$$v^2_{\text{interp}(s)} = v_i^2 + \lambda\,(v_{i+1}^2 - v_i^2), \qquad v_{\text{interp}(s)} = \sqrt{v_i^2 + \lambda\,(v_{i+1}^2 - v_i^2)}.$$

Compare this with the kinematic relation $v^2 = v_0^2 + 2a\,(s - s_0)$ from Section 3.1. They match term for term with

$$a = \frac{v_{i+1}^2 - v_i^2}{2\big(s_{i+1} - s_i\big)},$$

a **constant**. So interpolating in v^2 makes each keyframe interval a constant-acceleration segment — the same shape as every other limit in the planner.

3.7 Putting It Together: Take the Minimum

All five quantities are upper bounds on the speed at the current position, so the commanded speed is simply their minimum:

$$v_{\text{desired}} = \min\big\{v_{\max}, v_{\text{accel}}, v_{\text{brake}}, v_{\text{curve}}, v_{\text{interp}}\big\}.$$

The commanded angular velocity follows from the curvature at that point: $\omega = v_{\text{desired}} \cdot \kappa$.

3.8 Limitations of a Greedy Planner

The planner above is **greedy**: it looks only at the constraints at the robot's current position, plus the braking constraint toward the path's end. If a sharp turn or a slow keyframe lies ahead mid-path, nothing forces the robot to start braking for it early. By the time v_{curve} or v_{interp} drops, the robot may be physically unable to slow down fast enough, since real deceleration is also bounded by $a_{\max}$.

There is a second, milder approximation lurking in the same place. The constant-a distance formulas of Section 3.1 assume a clean trapezoidal profile: speed up, cruise at $v_{\max}$, slow down. On a curvy path the curvature limit often prevents the robot from ever reaching $v_{\max}$, so the profile gets cut off or dips in the middle, and the computed accelerate/brake distances slightly overestimate what is actually needed.

The standard fix addresses both. Precompute the profile over a discretized path with a **forward pass** that propagates acceleration limits forward and a **backward pass** that propagates deceleration limits

back from every slow point, then take the pointwise minimum. This two-pass idea goes back to the time-optimal path parameterization literature [6], [7], [8] and is exactly what tools like WPILib's trajectory generator do [4]; see Section 5.1.

4 Making It Real

4.1 Driving the Drivetrain

The planner emits a linear velocity v_{desired} and an angular velocity ω. Converting those to wheel commands for a differential drive is the inverse of the kinematics used in Section 3.5:

$$v_{\text{left}} = v_{\text{desired}} - \frac{\omega\, w}{2}, \qquad v_{\text{right}} = v_{\text{desired}} + \frac{\omega\, w}{2},$$

where w is the track width. Send these to your wheel velocity controllers.

Note that this is **open loop**: it assumes the robot ends up where the plan says it does. Wheel slip, unmodeled dynamics, and odometry drift all violate that assumption. To close the loop on pose feedback, wrap the output in a tracking controller such as pure pursuit [9] or RAMSETE.

4.2 Handling Multiple Path Segments

Many paths consist of several curve segments joined end-to-end. We apply the profiling approach to each segment in sequence, while ensuring smooth transitions between them. At the end of one segment the next begins with the robot continuing at whatever speed it reached; if the path was planned correctly, the two segments share a common point and velocity.

- If the remaining distance in the current segment is greater than Δs, the robot simply continues within the same segment.
- If Δs would carry the robot past the end of the current segment, move it to the end of that segment using the portion of Δs needed, then use the leftover distance to continue into the next segment (starting the next segment's arc length from 0).

The parameter t resets to 0 at the start of the new segment and the same calculations proceed there. If you are using keyframes, placing one at the segment boundary lets you enforce a specific speed at that point.

4.3 Code Walkthrough

The following is a simplified extraction from VMPLib. Every line corresponds to something derived above.

```
void TrapezoidalProfile::step() {
    ++step_count_;
    time_accum_ += dt_;
    s_current_ = sFunction(control_, prev_t_);

    float keyframe_lim  = computeKeyframeLimit();
    float curvature_lim = computeCurvatureVelocityLimit(prev_t_);
    float accel_lim     = computeAccelerationLimit();
    float brake_lim     = computeBrakingLimit(s_current_);

    // All limits are upper bounds; command the smallest.
    float desired_linear = std::min({ curvature_lim,
                                      accel_lim,
                                      brake_lim,
                                      keyframe_lim,
                                      max_lin_vel_ });
```

```
    float deltaS = desired_linear * dt_;
    float next_t = findNextT(s_current_, deltaS);

    float kappa = signedCurvature(control_, next_t);
    float turning_component = kappa * desired_linear;

    poses_.push_back(findXandY(control_, next_t));
    velocities_.push_back({ desired_linear, turning_component, time_accum_ });

    prev_t_    = next_t;
    cur_speed_ = desired_linear;
}
```

Each call advances the profile by one timestep and appends to the pose and velocity vectors rather than returning a sample; the caller loops on `isFinished()` and reads the accumulated trajectory afterwards. That termination test checks a step counter as well as $t \geq 1$, so a path that stalls near a cusp ends instead of spinning forever.

4.4 Ideas for Future Improvement

- **Motor model.** Acceleration decreases with speed, since motors produce less torque at high RPM. A first-order model is

$$a(v) = a_{\max}\left(1 - \frac{v}{v_{\text{free}}}\right),$$

  where v_{free} is the motor's free (no-load) speed.
- **Full dynamics.** Model mass and rotational inertia for more realistic acceleration behavior, possibly including friction and drag.
- **Backward propagation.** Implement the backward pass described in Section 3.8, propagating deceleration limits back from every slow point and taking the pointwise minimum. This is the single highest-value addition to the planner as it stands.

5 Related Work and Further Reading

5.1 Related Work and Similar Systems

The approach in this guide was developed independently for VMPLib, but each ingredient — and the overall pipeline — has well-established relatives worth knowing about.

Arc-length parameterization. Wang, Kearney, and Atkinson [2] describe essentially the same numerical machinery used here (Gauss–Legendre quadrature for $s(t)$, Newton–Raphson to invert it) for driving simulators, where vehicles must move along spline roads at prescribed speeds. Their paper is a good reference for accuracy and robustness details we glossed over.

Time-optimal path parameterization (TOPP). The academic formulation of "given a path and actuator limits, find the fastest feasible speed profile" dates to Bobrow et al. [6] and Shin and McKay [7], who proved the optimal profile alternates between maximum acceleration and maximum deceleration segments. Modern solvers such as TOPP-RA [8] compute the profile with a backward *controllable-set* pass followed by a forward pass — the rigorous version of the fix discussed in Section 3.8.

Spline trajectories for differential drives. Sprunk's thesis [10] is the closest academic sibling of this guide: it plans Bézier-spline paths for a differential drive robot and computes velocity profiles respecting

translational, rotational, and centripetal limits, with curvature-continuous joins between segments. See also the companion paper by Lau, Sprunk, and Burgard [11].

Competition robotics implementations. The FIRST Robotics Competition community has converged on the same architecture. Team 254's influential presentation *Motion Planning and Control in FRC* [12] popularized spline paths with trapezoidal profiles; WPILib's trajectory generator [4] performs a forward/backward pass subject to constraints, including a differential-drive wheel-speed constraint identical in spirit to our curvature limit; and GUI tools such as PathPlanner [13] and PATH.JERRYIO [5] provide the keyframe-editing workflow described earlier. Veness's free book [14] gives an accessible derivation of the RAMSETE controller mentioned in Section 4.1.

5.2 Further Learning Resources

- **YouTube — "The Continuity of Splines":** a deep dive into spline curves by Freya Holmér (highly recommended) [15]
- **Khan Academy:** derivatives and integrals (introductory calculus lessons)
- **Feynman Lectures on Physics**, Vol. 1: an insightful treatment of motion in Chapter 8
- **MIT OCW 18.01:** Single Variable Calculus (free online course materials)
- **MIT OCW 18.02:** Multivariable Calculus, for the material behind the curvature formula and the projection derivation

A Other Curve Representations

Cubic Béziers are used throughout this guide, but they are not the only option:

- **Quadratic Bézier curve.** Three control points (start, end, and one middle control point). Simpler, but less flexible in shaping the path.
- **Cubic Bézier curve.** Four control points (start, end, and two shaping points). More control over the shape; the choice made here.
- **Cubic Hermite spline.** Defined by endpoints and specified tangents at those endpoints, so you explicitly set the direction of travel at each end.
- **Catmull–Rom spline.** Passes through a series of given waypoints, smoothly interpolating all of them — convenient when the path must hit specific locations exactly.

Swapping representations requires changing only $\boldsymbol{r}(t)$, $\boldsymbol{r}'(t)$, and $\boldsymbol{r}''(t)$. Every other formula in this guide is written in terms of those three functions and is unaffected.

B Gauss–Legendre Nodes and Weights

Nodes x_i and weights w_i on the standard interval $[-1, 1]$ for $n = 2, 3, 4, 5$, as tabulated in standard references [16]. These are the constants plugged into the quadrature formula of Section 2.3; the $n = 5$ row is the one the implementation uses.

n	x_i (nodes)	w_i (weights)
2	$-\frac{1}{\sqrt{3}} \approx -0.577350$	1
	$+\frac{1}{\sqrt{3}} \approx 0.577350$	1
3	$-\sqrt{\frac{3}{5}} \approx -0.774597$	$\frac{5}{9} \approx 0.555556$
	0	$\frac{8}{9} \approx 0.888889$
	$+\sqrt{\frac{3}{5}} \approx 0.774597$	$\frac{5}{9} \approx 0.555556$
4	-0.861136	0.347855
	-0.339981	0.652145
	$+0.339981$	0.652145
	$+0.861136$	0.347855
5	-0.906180	0.236927
	-0.538469	0.478629
	0	$128/225 \approx 0.568889$
	$+0.538469$	0.478629
	$+0.906180$	0.236927

C Degenerate Cases and Cusps

Everything in this guide assumes the parametric speed $\|\boldsymbol{r}'(t)\|$ stays comfortably away from zero. Certain control-point placements violate that, producing a **cusp**: a point where the curve momentarily stops and reverses direction. Three separate pieces of machinery break there, all for the same reason.

Curvature is undefined. The denominator $\|\boldsymbol{r}'(t)\|^3$ in the curvature formula collapses to zero. It is tempting to return $\kappa = 0$ to avoid the division, but that is exactly backwards: a cusp is the **sharpest** point

on the path, geometrically a turn of zero radius, so $|\kappa| \to \infty$. Returning zero would remove the curvature speed limit precisely where it is needed most and send the robot through the cusp at full speed.

The safe convention is to saturate $|\kappa|$ at a large finite value, keeping the sign of the cross product wherever it is still recoverable so the turn direction survives. The limit $v_{\text{curve}} = v_{\max} \cdot R/(R + w/2)$ then drives $v \to 0$ as $R \to 0$, and since $\omega = v\kappa$ the angular velocity tends to $\omega \to 2v_{\max}/w$ — the robot turning in place at the drivetrain's maximum rate, which is the physically correct behavior at a cusp.

Newton's iteration for arc length breaks. Its derivative is $F'(t) = \|\boldsymbol{r}'(t)\|$, so the update step divides by something approaching zero and the iterate explodes. This is where the bisection fallback of Section 2.4 earns its place.

Quadrature accuracy degrades. The integrand loses smoothness near the cusp, so convergence slows and further subdivision is the remedy [2].

Practical advice. The cheapest defense is not to generate such paths in the first place — most path editors will show you the loop or the reversal visually — and to include a step-count cap in the termination test, so that a profile that stalls at a cusp terminates rather than hanging.

Bibliography

[1] G. Farin, *Curves and Surfaces for CAGD: A Practical Guide*, 5th ed. Morgan Kaufmann, 2002.

[2] H. Wang, J. Kearney, and K. Atkinson, "Arc-Length Parameterized Spline Curves for Real-Time Simulation," in *Proceedings of the 5th International Conference on Curves and Surfaces*, Saint-Malo, France, 2002, pp. 387–396. [Online]. Available: https://homepage.math.uiowa.edu/~atkinson/ftp/CurvesAndSurfacesArcLength.pdf

[3] S. M. LaValle, *Planning Algorithms*. Cambridge University Press, 2006. [Online]. Available: http://lavalle.pl/planning/

[4] WPILib, "Trajectory Generation and Constraints." [Online]. Available: https://docs.wpilib.org/en/stable/docs/software/advanced-controls/trajectories/trajectory-generation.html

[5] J. Lum, "PATH.JERRYIO: A Path Editor for VEX and FTC Robots." [Online]. Available: https://path.jerryio.com/

[6] J. E. Bobrow, S. Dubowsky, and J. S. Gibson, "Time-Optimal Control of Robotic Manipulators Along Specified Paths," *The International Journal of Robotics Research*, vol. 4, no. 3, pp. 3–17, 1985.

[7] K. G. Shin and N. D. McKay, "Minimum-Time Control of Robotic Manipulators with Geometric Path Constraints," *IEEE Transactions on Automatic Control*, vol. 30, no. 6, pp. 531–541, 1985.

[8] H. Pham and Q.-C. Pham, "A New Approach to Time-Optimal Path Parameterization Based on Reachability Analysis," *IEEE Transactions on Robotics*, vol. 34, no. 3, pp. 645–659, 2018, [Online]. Available: https://github.com/hungpham2511/toppra

[9] R. C. Coulter, "Implementation of the Pure Pursuit Path Tracking Algorithm," technical report CMU-RI-TR-92-01, 1992. [Online]. Available: https://www.ri.cmu.edu/pub_files/pub3/coulter_r_craig_1992_1/coulter_r_craig_1992_1.pdf

[10] C. Sprunk, "Planning Motion Trajectories for Mobile Robots Using Splines," Student project report, 2008. [Online]. Available: http://www2.informatik.uni-freiburg.de/~lau/students/Sprunk2008.pdf

[11] B. Lau, C. Sprunk, and W. Burgard, "Kinodynamic Motion Planning for Mobile Robots Using Splines," in *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2009, pp. 2427–2433.

[12] J. Russell, T. Bottiglieri, and A. Schuh, "Motion Planning and Control in FRC." [Online]. Available: https://www.youtube.com/watch?v=8319J1BEHwM

[13] M. Jansen, "PathPlanner: A Motion Profile Generator for FRC Robots." [Online]. Available: https://github.com/mjansen4857/pathplanner

[14] T. Veness, *Controls Engineering in the FIRST Robotics Competition*. Self-published, 2024. [Online]. Available: https://controls-in-frc.link/

[15] F. Holmér, "The Continuity of Splines." [Online]. Available: https://www.youtube.com/watch?v=jvPPXbo87ds

[16] M. Abramowitz and I. A. Stegun, *Handbook of Mathematical Functions with Formulas, Graphs, and Mathematical Tables*. in Applied Mathematics Series 55. National Bureau of Standards, 1964.